Project Documentation: RAG System with FastAPI and LangChain

1. Introduction

This project is a **Retrieval-Augmented Generation** (RAG) system built with FastAPI, LangChain, and OpenAI models. The RAG pattern combines information retrieval from a knowledge base with generative AI to provide more accurate and context-aware responses.

LangChain is a Python framework that helps integrate Large Language Models (LLMs) with external data sources, supporting features like document loading, text splitting, embeddings, and retrieval chaining. This makes it easier to build intelligent, document-aware AI applications.

Key Features

- Users can upload PDF, TXT, and DOCX documents.
- Documents are split into chunks, converted into vector embeddings, and stored in a vector database.
- On asking a question, the system retrieves relevant chunks and generates an answer using an LLM (GPT-4).
- Backend is powered by FastAPI for building scalable API endpoints.

---

2. Technologies and Libraries

Category	**Technology / Library**
Backend	FastAPI
Document Loading	PyPDFLoader, TextLoader, UnstructuredFileLoader
Text Splitting	RecursiveCharacterTextSplitter
Embeddings	OpenAIEmbeddings (text-embedding-ada-002)
Vector Storage	Chroma
LLM	ChatOpenAI (gpt-4o-mini)

Category	Technology / Library
Environment	Python 3.11+, dotenv
API Server	Uvicorn
Dependencies Mgmt	pip + requirements.txt

Installed Libraries (requirements.txt)

- **fastapi**
- **uvicorn**
- **python-dotenv**
- **langchain**
- **langchain-community**
- **langchain-openai**
- **chromadb**
- **pypdf**

---

3. Important Considerations

- **Python Installation**: Install Python 3.11+ (latest stable recommended).
- **Dependencies**: Install required libraries using pip install -r requirements.txt.
- **Supported File Types**: PDF, TXT, DOCX.
- **Chunking**: Adjust chunk_size and chunk_overlap based on document length.
- **OpenAI API Key**: Required and must be stored in .env.
- **Persistence**: Vector database persistence ensures data remains after a server restart.

---

4. Project Structure

```
project-root/
│
├─ docs/              # Folder for uploaded documents
├─ chroma_db/         # Vectorstore persistence folder
├─ main.py            # FastAPI application (core logic)
├─ .env               # Contains OPENAI_API_KEY
└─ requirements.txt   # Python dependencies
```

---

5. Architecture and Workflow

Textual Workflow

[User Uploads Document]

↓

[Document Loader] → [Text Splitter] → [Embeddings Generator] → [Vector Database (Chroma)]

↓

[User Asks Question]

↓

[Retriever fetches relevant chunks]

↓

[LLM generates answer]

↓

[Answer Returned to User]

Explanation:

- **Upload**: Users upload documents via the /upload endpoint.
- **Ingestion**: Documents are loaded using the correct loader based on file type.
- **Text Splitting**: Large documents are split into smaller chunks for semantic search.
- **Embedding**: Each chunk is converted to vector embeddings.
- **Vector Storage**: Chunks are stored in Chroma, enabling fast similarity-based retrieval.
- **Retrieval**: Relevant chunks are fetched when a question is asked.
- **Generation**: The LLM generates a context-aware answer.

---

6. Core Components

6.1 Document Loader

- Read different document types.
- **Code Example**:

```
if filename.lower().endswith(".pdf"):
    loader = PyPDFLoader(file_path)
elif filename.lower().endswith(".txt"):
    loader = TextLoader(file_path)
elif filename.lower().endswith(".docx"):
    loader = UnstructuredFileLoader(file_path)
```

- Choosing the correct loader ensures accurate text extraction.

6.2 Text Splitter

- Split large documents into chunks.
- **Code Example**:

```
splitter = RecursiveCharacterTextSplitter(chunk_size=800, chunk_overlap=200)
chunks = splitter.split_documents(all_docs)
```

- Parameters:
    - chunk_size: Max characters per chunk
    - chunk_overlap: Repeated characters between chunks to preserve context
- Maintains context and improves retrieval accuracy.

6.3 Embeddings & Vector Storage

- Convert chunks to vectors and store for semantic search.
- **Code Example**:

```
embeddings = OpenAIEmbeddings(model="text-embedding-ada-002")
vectorstore = Chroma(
    persist_directory="./chroma_db",
    embedding_function=embeddings
)
vectorstore.add_documents(chunks)
```

Enables retrieval of semantically similar content.

Step by Step Process:

- First, text is changed into numbers (vectors)
- A Chroma database is set up to save and organize these vectors.
    - persist_directory="./chroma_db" - where the data is saved on your computer.
    - embedding_function=embeddings - tells Chroma how to turn text into numbers.
- Finally, the document chunks are stored in Chroma so later you can find and match text by meaning.

6.4 Question-Answering Endpoint

- Accept a question and return an answer.
- **Code Example**:

```
retriever = vectorstore.as_retriever(search_kwargs={"k": 3})
llm = ChatOpenAI(model="gpt-4o-mini-2024-07-18", temperature=0)
qa_chain = RetrievalQA.from_chain_type(llm=llm, retriever=retriever)
result = qa_chain.run(question)
```

Core of RAG; ensures answers are accurate and context-aware.

Step by Step Process:

- Turns your vector database into a retriever that fetches top 3 relevant chunks.
- Loads the LLM (GPT-4o-mini) with deterministic answers (temperature=0).
- Builds a Q&A pipeline that combines retriever + LLM.
- ser question → retrieve docs → add context → LLM → answer returned.

---

7. API Endpoints

Endpoint	Method	Description
/upload	POST	Upload and ingest document (PDF, TXT, DOCX)
/ask/	POST	Ask a question and get an answer from uploaded documents

Example

- Upload a document:
    1. POST /upload
    2. file: example.pdf
- Ask a question:
    1. POST /ask/
    2. question: "What are the four stages of RAG?"
- **Response (JSON)**:


```
{
  "question": "What are the four stages of RAG?",
  "answer": "The four stages are: Ingestion, Storage, Retrieval, Generation."
}
```


---

8. Conclusion

This project demonstrates the power of Retrieval-Augmented Generation (RAG) by integrating document ingestion, vector-based semantic search, and large language models into one system. By using FastAPI, LangChain, and OpenAI models, it provides a scalable backend solution for building document-aware AI assistants.